

INTRODUCING NEW TRACING

Empowering Developers with Insightful Observability

16th June 2025

Jürgen Nicklisch

IOE, Performance & Tracing Team



STRUCTURE

1. Introduction
2. Coding
3. Configuration
4. Documentation and Consistency
5. Conclusions

INTRODUCTION

CARDANO-TRACER: MOTIVATION & OVERVIEW

Motivation

- Previously, the Cardano node handled all logging and monitoring responsibilities.
- It served metrics via two internal web servers: Prometheus and EKG.
- **cardano-tracer** decouples these concerns — reducing node complexity and resource usage.

System Overview

- Node: Produces logs and metrics.
- **cardano-tracer**: Consumes, processes, and optionally persists or forwards this data.
- Operates by periodically fetching and aggregating data from one or multiple nodes.

TRACE-DISPATCHER LIBRARY

- Central management and persistent activation of all tracers, respecting configured severity levels.
- Fine-grained configuration via hierarchical namespaces — down to individual trace messages.
- Transparent forwarding of trace data to **cardano-tracer**.
- Supports dynamic reconfiguration (hot-reload) after deprecation of legacy tracing.
- Generates documentation for all trace messages and metrics, automatically.
- Performs introspection-based sanity checks to validate tracer configurations and implementations.

LEGACY CONCEPTS RETAINED IN THE NEW TRACING SYSTEM

1. Typed trace messages remain central — modeled as algebraic data types (ADTs), with individual tracers defined per type.
2. In client code, emitting a trace remains simple: `traceWith tracer msgObj`.
3. Tracing parameters (e.g., formatting, naming, severity) are still defined in a centralized configuration section within the node codebase.
4. Continued use of the **contra-tracer** library for contravariant tracing — now integrated via an arrow-based implementation.

CODING

TRACING PRINCIPLES (HIGH LEVEL)

How to trace in Cardano (modern style):

1. Construct a Tracer

Each tracer is identified by a namespace (e.g., ["Startup"]).

It is essential that all tracers share consistent backend configuration.

2. Configure the Tracer

Configuration includes severity filtering, formatting, and routing.

Reflection-based mechanisms ensure that configuration is complete and correct.

3. Emit a Trace

Tracing is done by passing structured messages to the tracer.

The system captures structured events like "startup time" with timestamps.

Tracers are consistent, typed, and reflective.

CUSTOM TRACER EXAMPLE

```
-- A custom trace message type
data MyTracer = StartupTime UTCTime
               | EndTime UTCTime

-- Inside an IO context (e.g., main or an init function):
do
  trBase <- standardTracer
  trEkg <- ekgTracer trConfig ekgStore
  configReflection <- emptyConfigReflection
  startupTr <- mkCardanoTracer trBase (Just trEkg) ["Time"]
  configureTracers configReflection trConfig [startupTr]
  currentTime <- getCurrentTime
  traceWith startupTr (StartupTime currentTime)
```

LOGFORMATTING TYPECLASS

Purpose:

- Governs how trace messages are rendered for different audiences and systems.
- Enables both human-readable and machine-readable representations of trace events.

Core Capabilities:

- Machine-Readable Output: Required implementation that encodes the trace message as structured JSON, adaptable by detail level.
- Human-Readable Output: Optional textual rendering. Falls back to the machine form if left empty.
- Metric Extraction: Optional mechanism to extract 0 or more metrics from a trace message for monitoring purposes.

This typeclass bridges tracing, monitoring, and presentation in a unified abstraction.

LOGFORMATTING TYPECLASS

```
class LogFormatting a where
  forMachine :: DetailLevel -> a -> Aeson.Object

  forHuman :: a -> Text
  forHuman _v = ""

  asMetrics :: a -> [Metric]
  asMetrics _v = []

instance LogFormatting MyTracer where
  forMachine _detailLevel (StartupTime t) =
    mkObject [ "timestamp" .= (pack . show) t]
  forHuman (StartupTime t) = "Startup time: " <> (pack . show) t
```

NAMESPACES IN TRACING

`ChainDB.InitChainSelEvent.Validation.InvalidBlock`

- Hierarchical name used to identify trace messages.
- Facilitates organization, filtering, and routing of trace data.
- Empowers administrators to apply fine-grained control (e.g., severity, privacy).

Transition to Namespace-Only Identification

- Older system used both `Namespace` and `Kind` fields.
- New tracing deprecates `Kind` in favor of namespace-only identification.
- All tools and tracers should rely exclusively on the namespace.

NAMESPACE STRUCTURE

- System namespace: Top-level category (was `cardano.node`, now often empty).
- Prefix namespace: Passed explicitly when constructing a tracer (e.g., `["ChainDB"]`).
- Inner namespace: Determined by the `MetaTrace` instance (specific to the message).

```
data Namespace a = Namespace {  
    nsPrefix :: [Text],  
    nsInner  :: [Text]  
}
```

-- Example:

```
Namespace ["ChainDB"]  
    ["InitChainSelEvent", "Validation", "InvalidBlock"]
```

METATRACE TYPECLASS

Purpose:

- Supplies structured metadata about trace messages to enable filtering, documentation, severity management, and routing.
- Supports introspection and trace analysis without hardcoding details elsewhere.

Key Responsibilities:

- Namespaces: Defines trace classification hierarchies and message-specific tags.
- Severity Privacy: Determines the criticality and visibility (e.g., public vs. private) of each message.
- Detail Level: Specifies how much information is shown for a given message.
- Documentation: Enables optional descriptions for both messages and metrics to support introspection and observability tooling.

This typeclass enriches trace types with descriptive metadata.

METATRACE TYPECLASS

```
class MetaTrace a where
  namespaceFor  :: a -> Namespace a
  severityFor   :: Namespace a -> Maybe a -> Maybe SeverityS
  privacyFor    :: Namespace a -> Maybe a -> Maybe Privacy
  privacyFor _ _ = Just Public
  detailsFor    :: Namespace a -> Maybe a -> Maybe DetailLevel
  detailsFor _ _ = Just DNormal
  documentFor   :: Namespace a -> Maybe Text
  metricsDocFor :: Namespace a -> [(Text, Text)]
  metricsDocFor _ = []

  allNamespaces :: [Namespace a]
```

METATRACE INSTANCE

```
instance MetaTrace MyTracer where
  namespaceFor (StartupTime _) = Namespace [] ["Startup"]

  severityFor ns _ = case ns of
    Namespace [] ["Startup"] -> Just Info

  documentFor ns = case ns of
    Namespace [] ["Startup"] -> Just "Indicates startup time"

  allNamespaces = [Namespace [] ["Startup"]]
```


CONFIGURATION

TRACEOPTIONS OVERVIEW

Severity

- Filters messages based on importance.
- Levels: **Debug**, **Info**, **Notice**, **Warning**, **Error**, **Critical**, **Silence**

Detail Level

- Controls verbosity of message output.
- Levels: **DMinimal**, **DNormal**, **DDetailed**, **DMaximum**

Frequency Limiter

- Throttles messages by maximum frequency (e.g., 2.0 Hz)

Backends

- Specifies output destinations: **Stdout**, **EKGBackend**, **Forwarder**

HOW CONFIGURATION IS APPLIED

Namespace Hierarchy

- Configuration is applied per namespace.
- More specific paths override general ones.
- Example:
 - "" → global default
 - ChainDB → overrides global
 - ChainDB.Validation.InvalidBlock → most specific

Configuration File

- All settings are defined in a YAML or JSON file.
- Will support dynamic reloading at runtime in the future.

SEVERITY FILTERS

`TraceOptions:`

`"":`

`severity: Notice`

`ChainDB:`

`severity: Info`

`ChainDB.InitChainSelEvent.Validation.InvalidBlock:`

`severity: Silence`

- Each trace message has a default severity defined in code.
- Severity filters are configured using namespaces.
- More specific namespace rules override broader ones.
- Use `Silence` to suppress all messages from a namespace.

DETAIL LEVEL

TraceOptions:

"":

severity: Notice

detail: DNormal

- Available levels: DMinimal, DNormal, DDetailed, DMaximum.
- Applicable to messages that support multiple levels of verbosity.
- Enables refinement of output without changing message severity.
- Can be used to reduce noise or increase introspection granularity.

FREQUENCY LIMITERS

TraceOptions:

ChainDB.AddBlockEvent.AddedBlockToQueue:

```
# Only show a maximum of 2 of these messages per second  
maxFrequency: 2.0
```

- Frequency limiting is fully handled via configuration.
- Unlike legacy eliding tracers, no code-level changes are required.
- Limiting activity is itself traceable — limiter decisions are emitted.
- Helps reduce log spam from highly repetitive events.

BACKENDS FOR MESSAGES

Available Backends

- `Stdout` – console output in `MachineFormat` (JSON) or `HumanFormat`.
- `EKGBackend` – exposes metrics via EKG web server.
- `Forwarder` – sends messages to `cardano-tracer` using typed protocols.

TraceOptions:

"":

severity: Notice

detail: DNormal

backends:

- Stdout MachineFormat
- EKGBackend
- Forwarder

DOCUMENTATION AND CONSISTENCY

CONCLUSIONS

TIPS & TRICKS FOR DEVELOPERS

- Prefer Namespaces over Kind fields

The Kind field is deprecated — use structured namespaces for filtering and classification.

- Avoid Strictness Annotations

They offer no performance benefit for trace types and can actually degrade efficiency.

- Never Use **Show** Instances

Avoid using **Show** for trace message rendering. It creates unnecessary string allocations. Use **Text** or structured **Aeson** values instead.

- Talk to Us Early

Reach out to the Performance & Tracing Team for reviews, integration help, or questions.



PERFORMANCE & TRACING